Enzo Giaquinta, Agustin Contardi, Fernando Torrez, Ramiro Salcedo e Ignacio Sanchez

PDF TRABAJO UNIDAD N°7

PARTE 1:

Consigna 1:

- **¿Qué se hizo?** Se seleccionó el subconjunto relacional de pedidos y detalle de pedidos proveniente de schema.sql (pedido y detalle_pedido, junto con sus relaciones a cliente y producto).
- **¿Por qué se modeló así?** Siguiendo las recomendaciones de diseño en NoSQL, dado que las líneas de un pedido (detalle_pedido) no tienen vida independiente y siempre se consultan junto con el pedido, se decidió embeberlas en un arreglo (lineas[]) dentro de un único documento por pedido en la colección pedidos.
- **Identificadores y restricciones conservados:**
    - El _id del documento de MongoDB toma el valor del id_pedido original de PostgreSQL (garantizando trazabilidad).
    - Se conservan las referencias lógicas por ID (id_cliente y lineas[].id_producto).
    - Se respetan los tipos monetarios y de precisión mediante Decimal128 (equivalente a NUMERIC(10,2)).

Consigna 2:

- **¿Qué se hizo?** Se creó la base de datos food_store y la colección pedidos en MongoDB aplicando un esquema de validación estricto ($jsonSchema) para asegurar la integridad de los datos a nivel de base de datos.
- **Cumplimiento del "Alcance sugerido":** El trabajo práctico sugiere tomar un mínimo de cinco pedidos con sus detalles. Como se documenta en el script y en los datos de prueba (pedidos.json y migracion_mongodb.js), se incorporaron exactamente 5 pedidos coherentes con las reglas del dominio (con fechas UTC, estados válidos como CONFIRMADO y ENTREGADO, formas de pago, líneas de productos, cantidades positivas y precios unitarios históricos exactos).
- **Índices de rendimiento:** Se crearon índices compuestos y simples (id_cliente + fecha_pedido, lineas.id_producto, estado, fecha_pedido, monto_total) para garantizar velocidad en las consultas analíticas posteriores.

Consigna 3:

Indicar dos reglas de integridad que deben conservarse durante la migración y justificar brevemente qué datos se incluyeron dentro del documento en lugar de almacenarse en colecciones separadas.

Respuesta:

1. Dos reglas de integridad a conservar durante la migración:

1. **Inmutabilidad de precios y valores históricos:**
    - **Explicación:** Al registrar un pedido, los precios unitarios, subtotales y en el campo monto total, se debe reflejar exactamente el importe cobrado en el momento de la transacción. Si en el futuro los precios de los productos en la base de datos relacional (PostgreSQL) se modifican o actualizan, los datos guardados en la colección de MongoDB no deben sufrir cambios, preservando la integridad del historial financiero.
2. **Preservación de identificadores de origen y trazabilidad (_id, id_cliente):**
    - **Explicación:** Se conservan las claves primarias relacionales de PostgreSQL asociadas a los pedidos y clientes (mapeadas en los campos _id e id_cliente del documento). Esto garantiza la consistencia de datos entre ambos motores y asegura la trazabilidad al momento de comparar o auditar registros entre PostgreSQL y MongoDB.

2. Justificación de la inclusión de datos dentro del documento:

En lugar de crear colecciones separadas para clientes, pedidos y lineas_pedido, se optó por un modelo documental con datos embebidos, incorporando el detalle de las líneas y la información clave dentro del propio documento del pedido por las siguientes razones:

- **Optimización en el patrón de lectura:** La operación más frecuente en el dominio de Food Store es consultar el pedido completo. Al estar toda la información dentro del mismo documento, MongoDB recupera la orden en una sola lectura de disco, sin requerir de JOINS que disminuyan el rendimiento.
- **Atomicidad en las operaciones:** MongoDB garantiza que las actualizaciones e inserciones a nivel de un solo documento sean atómicas. Al tener los productos embebidos en el pedido, toda la transacción de la compra se guarda de forma consistente en un único paso sin riesgo de inconsistencias entre la cabecera y el detalle.

PARTE 2

Consigna 4:

Ejecutar en MongoDB las siguientes cuatro consultas, adaptadas a los nombres de campos del modelo implementado:

• a) Mostrar todos los pedidos, incluyendo su identificador y estado.
• b) Filtrar pedidos por un estado que exista en los datos migrados.
• c) Buscar pedidos que contengan un producto determinado dentro del arreglo de líneas.
• d) Buscar pedidos cuyo total sea superior a un importe elegido y justificar el umbral

Respuesta:

Consulta 1:
https://drive.google.com/drive/folders/1JFMhv9t2DbLGzj_6n8hcvRlUTlJw56dR?usp=sharing
Consulta 2:
https://drive.google.com/drive/folders/1GOL8-k4NMcTwny4DiHko8Pf2ChKStjam?usp=sharing
Consulta 3:
https://drive.google.com/drive/folders/18c2xjbc77ptOo1TFfrWwgkzgkE2BPge7?usp=sharing
Consulta 4:
https://drive.google.com/drive/folders/1NK2v4rVvLlclG7kMwOoQP3fOTpF0f54g?usp=sharing

Justificación del umbral de la consulta d:

Elegí el umbral de $1500 porque la distribución de montos de los datos migrados, están entre 1400 y 2200.50, ese límite me permite diferenciar las compras grupales o más caras de aquellas individuales menores

Consigna 5:

Consulta	Cantidad en PostgreSQL	Cantidad en MongoDB	Coinciden los IDs
Estado seleccionado (`ENTREGADO`)	3	3	Sí (IDs: `101`, `102`, `105`)
Producto seleccionado (`id_producto: 1`)	2	2	Sí (IDs: `101`, `103`)

Al ejecutar las consultas equivalentes en ambos motores de base de datos, se verifica una coincidencia del 100% tanto en la cantidad de registros recuperados como en los identificadores de los pedidos (`id_pedido`).

Filtro por estado ('ENTREGADO'): PostgreSQL recuperó los pedidos con ID `101`, `102` y `105`. La consulta sobre el campo raíz `estado` en MongoDB devolvió exactamente los mismos tres documentos.

Filtro por producto embebido (ID 1): En PostgreSQL se requirió realizar una combinación `INNER JOIN` entre las tablas `pedido` y `detalle_pedido` para filtrar por `id_producto = 1`, obteniendo los pedidos `101` y `103`. En MongoDB, la consulta utilizando `detalles.id_producto: 1` sobre el arreglo de documentos embebidos recuperó exactamente los mismos dos pedidos.

En conclusión no se detectaron diferencias entre los dos sistemas, lo que confirma que el proceso de migración conservó la integridad referencial y de datos exigida por las reglas de negocio de Food Store.

PARTE 3

Consigna 6:

Consulta realizada a la IA

Estamos modelando en MongoDB un subconjunto de Food Store que en PostgreSQL está implementado con las tablas pedido (cabecera: id_pedido, id_cliente, fecha, estado, monto_total), detalle_pedido (líneas: id_pedido, id_producto, cantidad, precio_unitario), cliente y producto. En MongoDB embebimos las líneas del pedido en un arreglo detalles[] dentro de un documento por pedido, conservando el id_pedido original como _id y los precios en Decimal128. El acceso más frecuente del sistema es consultar el pedido completo (cabecera + todas sus líneas + datos del cliente), y además necesitamos conservar la coherencia de cantidades y precios históricos: el precio registrado en el pedido no debe cambiar aunque el precio del producto se actualice. Compará PostgreSQL y MongoDB respecto del almacenamiento y la consulta de estos pedidos de Food Store: ¿cuál conviene para este subconjunto y por qué?

Respuesta de la IA

Criterio	**PostgreSQL (modelo actual)**	**MongoDB (modelo documental)**
Almacenamiento	Normalizado: pedido y detalle_pedido separados; integridad referencial garantizada con claves foráneas.	Documento único por pedido con detalles[] embebido; sin claves foráneas, la integridad se da por diseño y con reglas de validación ($jsonSchema).
Consulta del pedido completo	Requiere un JOIN entre pedido y detalle_pedido (y cliente) en cada lectura; costo crece con las líneas.	Una sola lectura del documento recupera el pedido completo, sin joins; el acceso frecuente a pedidos completos es más rápido y simple.
Coherencia de cantidades y precios	Se logra copiando el precio al detalle_pedido al confirmar el pedido (precio histórico); la actualización cabecera+detalle exige transacción ACID.	Al embeber líneas y cabecera en un documento, MongoDB garantiza atomicidad a nivel de documento: no puede quedar un detalle huérfano o desalineado del total.

Escalabilidad	Escala bien verticalmente; para lecturas masivas de pedidos completos, los joins limitan el rendimiento ante crecimiento de volumen.	Escala horizontalmente (particionamiento/sharding); el patrón de lectura de pedidos completos escala con mayor facilidad.

Conclusión parcial de la IA

Para el subconjunto pedidos con sus líneas, donde el 100% de las operaciones son lecturas de pedidos completos y los precios deben quedar congelados, el modelo embebido de MongoDB es más adecuado; sin embargo, si se requieren consultas analíticas que cruzan muchas entidades o integridad referencial estricta entre tablas, se justifica mantener PostgreSQL o combinar ambas tecnologías (PostgreSQL como fuente de verdad y MongoDB como repositorio de lectura).

Consigna 7:

Contraste de afirmaciones de la IA

Afirmación A1: «Al embeber líneas y cabecera en un documento, MongoDB garantiza atomicidad a nivel de documento: no puede quedar un detalle huérfano o desalineado del total.»

Contraste. La primera parte es correcta: la documentación oficial de MongoDB confirma que «an operation on a single document is atomic». Las operaciones sobre varios documentos requiere transacciones ACID multi-documento, disponibles solo en replica set o sharded cluster; un standalone no las soporta. Pero la afirmación va más allá de la atomicidad: que una escritura se aplique completa o no se aplique no implica que monto_total coincida con la suma de los subtotales. Un documento con un total mal calculado puede escribirse atómicamente y quedar inconsistente.

Evidencia del laboratorio. Cada pedido y sus líneas se guardan en un único documento (lineas[] embebido); migracion_mongodb.js (líneas 57–178) los inserta completos con insertMany(). No puede quedar una cabecera sin líneas ni líneas huérfanas. La atomicidad es por pedido: cada documento se inserta completo o no se inserta; el lote de insertMany() no es atómico en conjunto.

Veredicto: verdadera en la atomicidad por documento y en la ausencia de líneas huérfanas; falsa en la implicación de que el total queda alineado automáticamente.

Afirmación A2: «[…] sin claves foráneas, la integridad se da por diseño y con reglas de validación ($jsonSchema).»

Contraste. Es parcialmente verdadera. $jsonSchema es una validación declarativa real: exige campos, controla tipos BSON, rangos, enum y minItems, y con la acción de validación por defecto ("error") rechaza documentos inválidos. Pero no equivale a integridad «por diseño»: no puede expresar reglas cruzadas ni integridad referencial entre colecciones.

Evidencia del laboratorio. El validador de pedidos (migracion_mongodb.js, líneas 19–52) exige los campos principales, cantidad int con minimum: 1, precios y subtotales decimal con minimum: 0, estado y forma_pago por enum y al menos una línea. Rechaza cantidades nulas, precios negativos o estados inválidos.

Límites. $jsonSchema no puede expresar subtotal = cantidad × precio_unitario_facturado, monto_total = suma de subtotales, unicidad de id_producto dentro de lineas[] ni integridad referencial de id_cliente e id_producto contra otras colecciones. Esas reglas quedan en la ETL o en la aplicación (docs/esquema_nosql_pedidos.md, §4); PostgreSQL las declara con PK compuesta, claves foráneas y CHECK.

Veredicto: parcialmente verdadera. Acierta en que hay que validar y en que $jsonSchema ayuda, pero la integridad no se da sola por embeber: las reglas cruzadas y las referencias externas quedan fuera de la base documental..

Cuadro resumen del contraste

Afirmación	Evidencia principal	Veredicto
A1. Atomicidad y total alineado	Documentación de MongoDB: atomicidad por documento; las transacciones multi-documento requieren replica set o sharded cluster. Evidencia: lineas[] embebido, insertMany() por pedido.	Verdadera en atomicidad; falsa en alineación automática del total.
A2. Integridad por diseño + $jsonSchema	Documentación de MongoDB: $jsonSchema rechaza documentos inválidos. Evidencia: validador de migracion_mongodb.js, líneas 19–52; no cubre totales, unicidad ni FK.	Parcialmente verdadera; requiere ETL o aplicación.

Conclusión

Recomendación: combinar tecnologías, con PostgreSQL como fuente de verdad y MongoDB como modelo de lectura.

PostgreSQL debe conservar la escritura de pedidos, líneas, clientes y productos porque sus restricciones declarativas —claves foráneas, PK compuesta (id_pedido, id_producto), CHECK de cantidad y precio, NUMERIC(10,2)— garantizan la coherencia que el negocio necesita sin depender del código de la aplicación. **MongoDB** aporta valor para el acceso frecuente al pedido completo: al embeber las líneas en un solo documento, una lectura recupera la cabecera y el detalle sin JOIN y la escritura del pedido es atómica; además, el validador $jsonSchema controla tipos, rangos y estados.
Dos criterios técnicos sostienen la decisión:
- Integridad y transaccionalidad, donde PostgreSQL es más fuerte porque cubre reglas cruzadas que $jsonSchema no expresa
- Rendimiento de lectura y ajuste al patrón de acceso, donde el documento embebido simplifica y acelera la consulta de pedidos completos.

La combinación exige sincronización (outbox o captura de cambios), consistencia eventual y monitoreo; si hubiera que elegir un solo motor para este alcance, **PostgreSQL es la opción más segura. MongoDB se justificaría como almacén principal sólo si el volumen de lecturas y la disponibilidad pesaran más que la integridad inmediata.**

Referencias:

https://www.mongodb.com/docs/manual/core/transactions/
https://www.mongodb.com/docs/manual/core/transactions-production-consideration/
https://www.mongodb.com/docs/manual/core/schema-validation/
https://www.postgresql.org/docs/current/ddl-constraints.html